https://rupakm.github.io/leslie https://github.com/rupakm/leslie https://rupakm.github.io/leslie/docs

Verifying ABA with Leslie

Gaspard Reghem

July 2, 2026

Abstract

Our main objective is the verification of an implementation of Asynchronous Byzantine Agreement. The proof of correctness consists of building a simulation relation between the implementation and an implementation running in an idealised setting. The simulation ensures that the correctness of the idealised implementation implies the correctness of the original one. However, this implementation is complex; thus, building the simulation in one go is intractable. Fortunately, the implementation is designed as the interlocking of multiple sub-protocols (GBCA, WCC, RSD, Gather, AVSS, BRB). This architecture allows us to decompose the construction of the overall simulation into smaller steps. Each implementation of sub-protocols will be simulated by an idealised version. Thus, when building the simulation for a protocol using sub-protocols, the sub-implementations will be replaced by their idealised versions, making the construction easier.

0.1 Preliminaries

0.1.1 Traditional Framework

Definition 1 (Probabilistic Labelled Transition System)**.** PLTS

A probabilistic labelled transition system (PLTS) is a tuple $\mathcal{P} := (Q, q_\star, L, T)$ where Q is a (countable) set of states, $q_\star \in Q$ is the initial state, L is a (countable) set of labels and $T \subseteq Q \times (L \cup \{\tau\}) \times \mathcal{D}(Q)$ is a set of probabilistic transitions.

A partial execution of $\mathcal{P}$ is a sequence $q_0.l_0.q_1.l_1 \cdots \in (Q.L)^*.Q \cup (Q.L)^\omega$ such that, for every $n \in \mathbb{N}$, there exists $\mu_n \in \mathcal{D}(Q)$ with $(q_n, l_n, \mu_n) \in T$ and $q_{n+1} \in \bar{\mu_n}$. The set of finite executions is denoted $\mathcal{E}^*$. A partial execution is called an execution if $q_0 = q_\star$. The trace of a partial execution is its projection onto L. The set of traces of $\mathcal{P}$ is denoted $\mathcal{T}$. A scheduler s is a function from $\mathcal{E}^*$ to $\mathcal{D}(T \cup \{\bot\})$ such that, for all $e \in \mathcal{E}^*$ and $t \in \bar{s(e)}$, $t = \bot$ or $src(t) = last(e)$. The set of schedulers is denoted Sch.

Definition 2 (Partial Probabilistic Execution)**.** Partial Probabilistic Execution

Given $\mu \in \mathcal{D}(Q)$ and $s \in Sch$, one can define the partial probabilistic execution induced by (μ, s) as the function $\mathfrak{e}_{\mu,s}$ from $\mathcal{E}^*$ to $[0, 1]$ such that if $e \in Q$ then $\mathfrak{e}_{\mu,s}(e) = \mu(e)$; otherwise, $e = e'.l.q$ and $\mathfrak{e}_{\mu,s}(e) = \mathfrak{e}_{\mu,s}(e') \cdot \sum_{t \in T | lbl(t) = l} s(e')(t) \cdot out(t)(q)$.

A partial probabilistic execution is called a probabilistic execution if it is induced by a pair (μ, s) such that $\mu = \delta_{q_\star}$. If $s \in Sch$ terminates almost surely from μ (i.e. $\sum_{e \in \mathcal{E}^*} \mathfrak{e}_{\mu,s}(e) \cdot s(e)(\bot) = 1$) then $\mathfrak{q}_{\mu,s} = last^{-1} \circ \mathfrak{e}_{\mu,s}$ is a well-defined probability distribution over states. We denote $\mu l \mu'$ if and only if there exists $s \in Sch$ such that $\delta_l = \mathfrak{t}_{\mu,s}$ and $\mu' = \mathfrak{q}_{\mu,s}$. Partial probabilistic executions induce a measure on the σ-field induced by cones of partial executions, and the trace function is measurable from the σ-field induced by cones of partial executions to the one induced by cones of traces. Thus, $\mathfrak{t}_{\mu,s} = tr^{-1} \circ \mathfrak{e}_{\mu,s}$ is well-defined and called a trace distribution. The set of trace distributions achievable by $\mathcal{P}$ is defined as those induced by a probabilistic execution of $\mathcal{P}$ and denoted $\mathrm{tdist}(\mathcal{P})$. A hyper-transition of a PLTS $\mathcal{P}$ is a triple (μ, l, μ') such that, for all $q \in \bar{\mu}$, there exists $(q, l, \mu_q) \in T$ such that $\mu' = \sum_{q \in Q} \mu(q) \cdot \mu_q$. The set of hyper-transitions is denoted H. The trace distribution pre-order $\mathcal{P} \subseteq^t \mathcal{P}'$ is defined by $\mathrm{tdist}(\mathcal{P}) \subseteq \mathrm{tdist}(\mathcal{P}')$.

Definition 3 (Parallel Composition of PLTS)**.** Parallel Composition

Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$ and $S \subseteq L \cap L'$, the composed PLTS $\mathcal{P} \parallel_S \mathcal{P}'$ is defined as $(Q \times Q', (q_\star, q'_\star), L \cup L', T \parallel_S T')$ where $((q, q'), l, \mu'') \in T \parallel_S T'$ if and only if

- $l \in S$ and there exist $(q, l, \mu) \in T$ and $(q', l, \mu') \in T'$ such that $\mu \otimes \mu' = \mu''$; or
- $l \notin S$ and there exists $(q, l, \mu) \in T$ such that $\mu \otimes \delta_{q'} = \mu''$; or

- $l \notin S$ and there exists $(q', l, \mu') \in T'$ such that $\delta_q \otimes \mu' = \mu''$.

The trace distribution pre-order is not stable under $\|_S$: $\mathcal{P} \subseteq^t \mathcal{P}'$ does not imply $\mathcal{P} \|_S \mathcal{P}'' \subseteq^t \mathcal{P}' \|_S \mathcal{P}''$ [Seg95]. Therefore, the largest pre-order included in $\subseteq^t$ stable under $\|_S$ is considered: $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ if and only if, for all PLTSs $\mathcal{P}''$ and $S \subseteq L \cap L' \cap L''$, $\mathcal{P} \|_S \mathcal{P}'' \subseteq^t \mathcal{P}' \|_S \mathcal{P}''$.

Lemma 1. *If $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ then, for all PLTSs $\mathcal{P}''$ and $S \subseteq L \cap L' \cap L''$, $\mathcal{P} \|_S \mathcal{P}'' \sqsubseteq^t \mathcal{P}' \|_S \mathcal{P}''$*

Proof. This is a direct consequence of the associativity of $\|_S$. □

Definition 4 (Abstraction Operator)**.** Given a PLTS $\mathcal{P}$ and $I \subseteq L$, the abstract PLTS $\tau_I(\mathcal{P})$ is obtained by replacing all labels in I by τ.

Lemma 2. *If $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ then, for all $I \subseteq L$, $\tau_I(\mathcal{P}) \sqsubseteq^t \tau_I(\mathcal{P}')$.*

Given $R \subseteq A \times \mathcal{D}(B)$, the probabilistic lifting of R, denoted $\mathcal{D}(R)$, is defined by $(\mu_A, \mu_B) \in \mathcal{D}(R)$ if and only if there exists a witness $w : R \to [0, 1]$ such that $\mu_A = a \mapsto \sum_{\mu \in \mathcal{D}(B)} w(a, \mu)$ and $\mu_B = \sum_{(a,\mu) \in R} w(a, \mu) \cdot \mu$.

Definition 5 (Probabilistic Forward Simulation)**.** Probabilistic Forward Simulation

Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times \mathcal{D}(Q')$ such that $(q_\star, \delta_{q'_\star}) \in \mathcal{S}$ and, for all $(q, \mu') \in \mathcal{S}$ and $(q, l, \mu) \in T$, there exists $\mu' l \mu''$ such that $(\mu, \mu'') \in \mathcal{D}(\mathcal{S})$.

Theorem 3 (from [Seg95])**.** *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ if and only if there exists a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. The proof is non-trivial and is detailed in Segala's PhD thesis. Our approach to this proof is detailed in Appendix **??**. □

0.1.2 Non-probabilistic Systems

Non-probabilistic systems can be considered as special instances of PLTSs in which all transitions lead to a Dirac distribution. By identifying each Dirac distribution with the unique state in its support, we may assume that non-probabilistic systems are modelled by LTSs, which leads to several simplifications. In particular, probabilistic forward simulation is equivalent to a simpler simulation relation. On an LTS, we write qlq' if there exists a partial execution from q to q' whose trace is l.

Definition 6 (Weak Simulation)**.** Weak Simulation

Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times Q'$ such that $(q_\star, q'_\star) \in \mathcal{S}$ and, for all $(q, q') \in \mathcal{S}$ and $(q, l, p) \in T$, there exists $q' l p'$ such that $(p, p') \in \mathcal{S}$.

Lemma 4. *Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ if and only if there exists a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. According to Theorem 3, it suffices to prove that there exists a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ if and only if there exists a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$. For one direction, it suffices to note that any weak simulation between LTSs is a probabilistic forward simulation. For the other direction, if $\mathcal{S}$ is a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ then $\{(q, q') \mid \exists (q, \mu) \in \mathcal{S}, q' \in \bar{\mu}\}$ is a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$. □

There is a natural injection of LTSs into PLTSs, but also a natural projection of PLTSs onto LTSs. It consists in transforming all probabilistic non-determinism into classical non-determinism. The projection of a PLTS $\mathcal{P}$, denoted $\rho(\mathcal{P})$, is the LTS whose transitions are defined by $\{(q, l, q') \mid \exists \mu \in \mathcal{D}(Q), (q, l, \mu) \in T \wedge q' \in \bar{\mu}\}$. ρ is trivially the identity on LTSs, and ρ preserves the set of executions and hence the set of traces. As a result, the notion of weak simulation can be lifted to PLTSs: $\mathcal{S}$ is a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ if and only if it is a weak simulation of $\rho(\mathcal{P})$ by $\rho(\mathcal{P}')$. Since ρ is a projection onto LTSs, this is consistent with the previous definition.

0.1.3 Preserving Liveness

Liveness properties should not be tested with respect to the set of trace distributions and the set of traces of a PLTS because these sets are downward-closed and, in particular, they respectively always contain the Dirac distribution of the empty trace and the empty trace. Therefore, against such sets, liveness properties like termination are not satisfied by any PLTS. This may seem absurd but it hints at an implicit progress assumption made when evaluating liveness properties. This progess assumption could be stated as: if a transition is enabled then a transition must fire. Thus, it discards all executions that are finite and whose last state is not a deadlock. Executions satisfying the progress assumptions are said to be maximal and traces of maximal executions are also called maximal. Non-probabilistic liveness properties should tested with respect to the set of maximal traces. Similarly, a scheduler is said to be maximal if it schedules $\perp$ with positive probability only from maximal executions and a trace distribution is said maximal if it is induced by a maximal scheduler. Probabilistic liveness properties should be tested against maximal trace distribution.

In the setting of distributed systems suffering from Byzantine failures, another assumption is necessary in order to define a sensible evaluation of liveness properties. Indeed, since Byzantine processes behave arbitrarily, they can monopilise the execution preventing correct processes from taking a step. As a result, a fairness assumption is introduced stating that, in any infinite execution, correct processes must be scheduled infinitely often. To model it formally, a fairness function $\text{fair} : T \to \mathbb{B}$ is introduced which indicates which transitions are fair (e.g. transition of correct processes) and which are unfair (e.g. transitions of Byzantine processes). However, Byzantine failures also interfere with the notion of deadlock as they are allowed to crash at any time. We call a fair deadlock any state whose set of enabled transitions is included in the set of unfair transition. An execution is said to be fair if (a) it is finite and end in a

fair deadlock, or (b) it contains infinitely many fair transitions. A trace is said fair if it is induced by a fair execution. Non-probabilistic liveness properties will be evaluated with respect to the set of fair traces. Similarly, a scheduler is fair if (a) it schedules $\bot$ with positive probability only from fair executions and (b) all infinite executions consistent with it are fair. A trace distribution is fair if it is induced by a fair scheduler. Probabilistic liveness properties will be evaluated against the set of fair trace distribution.

The set of fair traces is denoted $\mathcal{T}^f$ and the set of fair trace distribution is denoted $\text{tdist}^f(\mathcal{P})$. Unfortunately, inclusion of $\mathcal{T}^f$ is not preserved by parallel composition because synchronisation issue could create new deadlocks. However, in our case, it never happens because synchronisation will be used to call sub-processes locally. The modelling solution is to use the input-output approach of [LT26] where the set of labels is partionned between input labels In and output labels Out. We require that the input labels are always enabled: $\forall q \in Q, l \in In, \exists \mu \in \mathcal{D}(Q), (q, l, \mu) \in T$. This requirement is easily fulfilled by adding self-looping transition labelled by each input label to all states. Moreover, we restrict composition to PLTSs whose outputs labels are distinct. This restrcition is not problematic either because labels will encode the protocol APIs. Under such assumptions, parallel composition does not create new deadlocks and so the inclusion of $\mathcal{T}^f$ is preserved by composition. As for $\subseteq^t$, inclusion of tdist is not preserved by parallel composition, and the problem cannot be solved by harmless restrictions. Thus, the fair trace distribution pre-congruence is defined as:

$$\mathcal{P} \sqsubseteq^f \mathcal{P}' \iff \forall \mathcal{P}'', S \subseteq L, \text{tdist}^f(\mathcal{P} \parallel_S \mathcal{P}'') \subseteq \text{tdist}^f(\mathcal{P}' \parallel_S \mathcal{P}'')$$

It beneficiates from the same algebraic properties as $\sqsubseteq^f$.

Probabilistic forward simulation and weak simulation preserve $\sqsubseteq^t$, but not $\sqsubseteq^f$. Looking at the folklore construction made in Lemma 4, a fair execution can be simulated by a unfair execution. If the execution ends with a fair deadlock then the simulation does not guarantee anything about the last state of the simulating execution. Moreover, internal fair transitions can be elided, thus, a infinite fair execution can be simulated by an execution with finitely many fair transitions. We call an exection a fair divergence if it is fair and its trace is τ. The diagnostic is the following: both simulations do not simulate fair divergences properly. We will say that a weak simulation $\mathcal{S}$ is fair if, for all $(q, p) \in \mathcal{S}$, if q fairly diverges then p fairly diverges. However, this simple solution has a major problem: proving that a state does not fairly diverge is hard. Fortunately, [DSW22] proposes a solution to a similar problem: equip the simulation with a ranking function. We adapt their contribution to our setting.

Definition 7 (Fair Weak Simulation)**.** FairWeakSimulation

Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, a fair weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times Q'$ equipped with a well-founded preorder $\leq$ on Q such that $(q_\star, q'_\star) \in \mathcal{S}$ and, for all $(q, q') \in \mathcal{S}$ and $(q, l, p) \in T$, there exists $q' l p'$ such that $(p, p') \in \mathcal{S}$. Furthermore, if $l = \tau$ and $q' \tau p'$ contain no fair transition then if (q, l, p) is fair then $q > p$; else $q \geq p$. Moreover, for all $(q, q') \in \mathcal{S}$, if q is a fair deadlock then q' fairly diverges.

For probabilistic forward simulation, the problem is more complicated because the construction of the simulating scheduler is not so clear (see Appendix **??**). However, the core idea stays the same even though it is more convoluted.

Definition 8 (Fair Probabilistic Forward Simulation)**.** FairProbForwardSimulation

Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, a fair probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times \mathcal{D}(Q')$ equipped with a well-founded preorder $\leq$ on Q such that $(q_\star, \delta_{q'_\star}) \in \mathcal{S}$ and, for all $(q, \mu') \in \mathcal{S}$ and $(q, l, \mu) \in T$, there exists $\mu' l \mu''$ such that $(\mu', \mu'') \in \mathcal{D}(\mathcal{S})$. Furthermore, let s be the scheduler inducing $\mu' \tau \mu''$ and w the witness of $(\mu', \mu'') \in \mathcal{D}(\mathcal{S})$, if $l = \tau$ and there exist $q' \in \bar{\mu}' \tau p'$ consistent with s which contain no fair transition then, for all $p \in \bar{\mu}$, if there exists $\mu_p \in \mathcal{D}(Q')$ such that $w(p, \mu_p) > 0$ and $\mu_p(p') > 0$ then if (q, l, μ) is fair then $q > p$; else $q \geq p$. Moreover, for all $(q, \mu') \in \mathcal{S}$, if q is a fair deadlock then there exists a fair scheduler s such that $\mathfrak{t}_{\mu', s} = \delta_\tau$.

Theorem 5. *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, if there exists a fair probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ then $\mathcal{P} \sqsubseteq^f \mathcal{P}'$.*

Proof. See Appendix **??**. □

Lemma 6. *Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, if there exists a fair weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ then $\mathcal{P} \sqsubseteq^f \mathcal{P}'$.*

Proof. It suffices to note that a fair weak simulation between LTSs is a fair probabilistic forward simulation. □

0.1.4 Partial Information

We encode partial information by restrictions on the set of schedulers. Intuitively, if the scheduler does not have access to all information then it might not be able to distinguish some states or some labels; thus, some distinct executions might look the same to it and it should schedule the same probability distribution of transitions.

Definition 9 (Adversary)**.** An adversary is a couple of observation function $(\mathcal{O}_Q, \mathcal{O}_L)$ which respectively map states and labels to state observation and label observation such that, for all $t, t' \in T$, if $src(t) = src(t')$ and $\mathcal{O}_L(lbl(t)) = \mathcal{O}_L(lbl(t'))$ then $t = t'$.

The injectivity of $\mathcal{O}_L$ on the set of transitions enabled in a given state ensures that partial information does not prevent the adversary from resolving nondeterminism. An adversary $(\mathcal{O}_Q, \mathcal{O}_L)$ induces an observation $\mathcal{O}_\mathcal{E}$ on executions defined by $\mathcal{O}_\mathcal{E}(q_0.l_0 \cdots) = \mathcal{O}_Q(q_0).\mathcal{O}_L(l_0) \cdots$. A scheduler s is consistent with an adversary if, for all $e, e' \in \mathcal{E}^*$, $\mathcal{O}_\mathcal{E}(e) = \mathcal{O}_\mathcal{E}(e') \implies s(e) = s(e')$. The omniscient adversary is encoded by setting the observation functions to the identity.

Our simulations are sound for partial-information adversaries because they were designed to be sound for omniscient adversaries. However, there is a completeness gap as partial information makes more reductions sound. Our chosen solution is to build a PLTS on which the omniscient adversary is equivalent to the partial-information adversary on the original PLTS. This is the belief construction.

Definition 10 (Belief PLTS)**.** Given a PLTS $\mathcal{P}$ and an adversary $\mathcal{A} := (\mathcal{O}_Q, \mathcal{O}_L)$, the belief PLTS $\mathcal{P}_\mathcal{A}$ is defined as the tuple $(\{\mu \in \mathcal{D}(Q) \mid \mathcal{O}_Q(\bar{\mu}) = \{\cdot\}\}, \delta_{q_\star}, \mathcal{O}_L(L), T_\mathcal{A})$ where

$$\mu \xrightarrow{l} \mu' \in T_\mathcal{A} \iff \exists \mu \xrightarrow{l} \mu'' \in H, \mu' = (q \mapsto \frac{\not\Vdash_{\mathcal{O}_Q(q)=o} \cdot \mu''(q)}{\mu''(\{q \mid \mathcal{O}_Q(q) = o\})}) \mapsto \mu''(\{q \mid \mathcal{O}_Q(q) = o\})$$

Theorem 7. *Given a PLTS $\mathcal{P}$ and an adversary $\mathcal{A}$, the set of trace distributions of $\mathcal{P}$ induced by a scheduler consistent with $\mathcal{A}$ is equal to tdist($\mathcal{P}$).*

When considering the composition of two PLTSs $\mathcal{P}$ and $\mathcal{P}'$ scheduled by their respective adversaries $\mathcal{A}$ and $\mathcal{A}'$, we need to define a composite adversary. Given $\mathcal{P}$ and $\mathcal{P}'$ two PLTSs and $\mathcal{A}$ and $\mathcal{A}'$ two adversaries (one for each PLTS) and $S \subseteq L \cap L'$, we say that $\mathcal{A}$ and $\mathcal{A}'$ are compatible on S if (i), for all $l \in S$, $\mathcal{O}_L(l) = \mathcal{O}'_L(l)$; (ii) $\mathcal{O}_L(L \setminus S) \cap \mathcal{O}_L(S) = \emptyset$ and (iii) $\mathcal{O}'_L(L' \setminus S) \cap \mathcal{O}'_L(S) = \emptyset$. The three requirements state that the adversaries of each PLTS get the same information from the synchronised labels and distinguish synchronised labels from the others. If $\mathcal{A}$ and $\mathcal{A}'$ are compatible on S, we can define $\mathcal{A} \parallel_S \mathcal{A}'$ as

$$((q, q') \mapsto (\mathcal{O}_Q(q), \mathcal{O}'_Q(q')), l \mapsto \begin{cases} \mathcal{O}_L(l) \text{ if } l \in L \\ \mathcal{O}'_L(l) \text{ if } l \in L' \end{cases})$$

Theorem 8. *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$ and their respective adversary $\mathcal{A}$ and $\mathcal{A}'$ and $S \subseteq L \cap L'$, if $\mathcal{A}$ and $\mathcal{A}'$ are compatible on S then $\mathcal{P}_\mathcal{A} \parallel_S \mathcal{P}'_{\mathcal{A}'}$ and $(\mathcal{P} \parallel_S \mathcal{P}')_{\mathcal{A} \parallel_S \mathcal{A}'}$ are isomorphic.*

Proof. For clarity, let us denote $\mathcal{P}_1 := \mathcal{P}_\mathcal{A} \parallel_S \mathcal{P}'_{\mathcal{A}'}$ and $\mathcal{P}_2 := (\mathcal{P} \parallel_S \mathcal{P}')_{\mathcal{A} \parallel_S \mathcal{A}'}$. First, note that Q_1 is in $\mathcal{D}(Q) \times \mathcal{D}(Q')$ whereas Q_2 is in $\mathcal{D}(Q \times Q')$ and there is no bijection between the two. However, one can prove that Q_2 is actually included in $\{\mu \in \mathcal{D}(Q \times Q') \mid \mu := \mu' \otimes \mu''\}$, thus, $h : (\mu, \mu') \mapsto \mu \otimes \mu'$ is a bijection between Q_1 and Q_2. It remains to prove the morphism part which consists in unfolding definitions. □

0.2 List of the Protocols

Protocols will be introduced as a set of properties to satisfy given in natural language. Then, an implementation taken from the literature will be presented as pseudo-code. Finally, a PLTS encoding of these properties will be provided to which we aim to reduce the implementation. For readability, pseudo-code and PLTS encoding will be relegated to Appendix.

Common Notation. Without loss of generality, we can assume that each process has a unique identifier ranging from 1 to n. The set of identifiers is denoted $[n]$. The number of faulty processes is bounded by a threshold denoted f and the set of faulty processes is denoted F.

0.2.1 Asynchronous Byzantine Agreement (ABA)

ABA is a canonical protocol where processes must agree on a common output. Processes start with a bit $\{0, 1\}$ and may return a bit. Its interface is composed of a *call* and a *return* statement. A program implements ABA if it satisfies three properties: Validity, Agreement, Almost-Sure Termination.

- Validity: if a correct process returns b then a correct process had b as input.
- Agreement: if two correct processes return, then their outputs are equal.
- Almost-sure Termination: if $n - f$ correct processes take part in the protocol then, with probability 1, all correct processes will eventually return.

Implementation 1 (from [ABDY22])**.** This implementation relies on GBCA and WCC. It proceeds in asynchronous rounds in which processes start by performing GBCA, then call WCC. They use their input as for the first round then update it based on the outcome of GBCA and WCC. The pseudo-code is provided in Algorithm **??**.

Definition 11. ABA.Core
The PLTS encoding Algorithm **??** is denoted ABA.Core. Its adversary is omniscient.

Specification 1. ABA.Spec
The PLTS encoding of ABA is provided in Transition System **??**. We call it ABA.Spec and its adversary is omniscient. It contains a probabilistic transition whose outcome depends on an unspecified ϵ which corresponds to the goodness of WCC.

Definition 12. ABA.Impl
ABA:Core, GBCA:Impl, WCC:Impl ABA.Impl is defined as the parallel composition of ABA.Core, GBCA.Impl and WCC.Impl synchronised on calls, returns and failures from which the API of GBCA and WCC have been abstracted. Since the adversaries of ABA.Core and GBCA.Impl are omniscient and the adversary of WCC.Impl is omniscient on its interface, the composed adversary is well-defined.

Definition 13. ABA.Hybrid
ABA:Core, GBCA:Spec, WCC:Spec ABA.Hybrid is defined as the parallel composition of ABA.Core, GBCA.Spec and WCC.Spec synchronised on calls, returns and failures from which the API of GBCA and WCC have been abstracted. Since all of their adversaries are omniscient, the composed adversary is well-defined and omniscient.

0.2.2 Graded Binding Crusader Agreement (GBCA)

Processes start with a binary input $\{0, 1\}$ and may return a value in $\{(0, A), (0, B), (\perp, C), (1, B), (1, A)\}$. A program implements GBCA if it satisfies four properties: Validity, Graded Agreement, Binding, Termination.

- Validity: if a correct process does not return (b, A) then a correct process had $1 - b$ as input.
- Graded Agreement: if two correct processes return (b, X) and (b', X') then $b = 0 \implies b' \neq 1$ and $X = A \implies X' \neq \perp$.
- Binding: when a correct process returns then a bound value b is defined and, in all extensions of the execution, correct processes do not return $(1 - b, B)$ or $(1 - b, A)$.
- Termination: if $n - f$ correct processes take part in the protocol then all correct processes will eventually return.

Implementation 2 (from [ABDY22])**.** This implementation consists of 4 rounds of message exchange. It does not rely on any sub-protocol and its algorithmic core idea is quorum intersection. The pseudo-code can be found in Algorithm **??**.

Definition 14. GBCA.Impl

The PLTS encoding Algorithm **??** is denoted GBCA.Impl. Its adversary is omniscient.

Specification 2. GBCA.Spec

The PLTS encoding of GBCA is provided in Transition System **??**. It is called GBCA.Spec and its adversary is omniscient. Since the branching property is not preserved by our simulation, we enhance the interface of GBCA to include the bound value in the return labels. This way, the binding property becomes linear.

0.2.3 Weak Common Coin (WCC)

Processes start with no input and may return a bit. A program implements WCC if it satisfies three properties: ϵ-Goodness, Unpredictability, Termination.

- ϵ-Goodness: with probability ϵ, all correct processes return 0 (resp. 1).
- Unpredictability: until $f + 1$ processes have called WCC, all outcomes are still possible.
- Termination: if $n - f$ correct processes take part in the protocol then all correct processes will eventually return.

Implementation 3 (simplified from [FKT22])**.** The implementation of WCC relies on SRSD and Gather. The idea is to randomly assign a number to each process then select a subset of processes and return based on the value assigned to these processes.

Definition 15. WCC.Core

The PLTS encoding Algorithm **??** is denoted WCC.Core. Its adversary is omniscient.

Specification 3. WCC.Spec

The PLTS encoding of WCC is provided in Transition System **??**. It is called WCC.Spec and its adversary is omniscient. In order to preserve unpredictability, we enhance the interface with a new guess instruction which allows the adversary to make a guess about the future return value. Unpredictability states that this guess cannot be almost-surely correct.

Definition 16. WCC.Impl

WCC:Core, Gather:Impl, SRSD:Impl WCC.Impl is defined as the parallel composition of WCC.Core, SRSD.Impl and Gather.Impl synchronised on calls, returns and failures from which the API of SRSD and Gather have been abstracted. Since the adversaries of WCC.Core and Gather.Impl are omniscient and the adversary of SRSD.Impl is omniscient on its interface, the composed adversary is well-defined.

Definition 17. WCC.Hybrid

WCC:Core, Gather:Spec, SRSD:Spec WCC.Hybrid is defined as the parallel composition of WCC.Core, Gather.Spec and SRSD.Spec synchronised on calls, returns and failures from which the API of SRSD and Gather have been abstracted. Since all of their adversaries are omniscient, the composed adversary is well-defined and omniscient.

0.2.4 Gather

Processes start with an input $x \in X$ and may return a partial function $f : Proc \rightharpoonup X$ presented as a subset of $Proc \times X$. A program implements Gather if it satisfies four properties: Validity, Agreement, Binding Common Core, Termination.

- Validity: if a correct process returns f and $f(q)$ is defined and q is correct then $f(q)$ is q's input.
- Agreement: if two correct processes return f and g then $f(q)$ and $g(q)$ are equal or one of them is undefined.
- Binding Common Core: once a correct process has returned, there exists $S \subseteq Proc$ of size at least $n - f$ such that every f returned by a correct process in the future is defined on S.
- Termination: if $n - f$ correct processes take part in Gather then all correct processes will eventually return.

Implementation 4 (taken from [AFW25])**.** This implementation relies on BRB and is non-probabilistic. Basically, each process will broadcast its input and then three rounds of message exchange allow processes to decide which broadcasted inputs they should return.

Definition 18. Gather.Core

The PLTS encoding Algorithm **??** is denoted Gather.Core. Its adversary is omniscient.

Specification 4. Gather.Spec

The PLTS encoding of Gather is provided by Transition System **??**. It is denoted Gather.Spec and its adversary is omniscient. Similarly to GBCA, Binding Common Core is transformed into a linear property by enhancing the *return* labels with the bound value.

Definition 19. Gather.Impl Gather:Core, BRB:Impl Gather.Impl is defined as the parallel composition of Gather.Core and BRB.Impl synchronised on calls, returns and failures from which the API of BRB has been abstracted. Since all of their adversaries are omniscient, the composed adversary is well-defined and omniscient.

Definition 20. Gather.Hybrid Gather:Core, BRB:Spec Gather.Hybrid is defined as the parallel composition of Gather.Core and BRB.Spec synchronised on calls, returns and failures from which the API of BRB has been abstracted. Since all of their adversaries are omniscient, the composed adversary is well-defined and omniscient.

0.2.5 Single Random Secret Draw (SRSD)

The SRSD protocol is actually a pair of protocols: Assignment Asg and Reconstruction Rec. For each instance, a process is designated beforehand as the owner; all processes know who the owner is. Assignment assigns a random value to the owner such that the assigned value is unknown to all. Reconstruction reveals the assigned value to processes. A correct implementation of SRSD should satisfy five properties.

- Assignment Termination: if all correct processes call SRSD.Asg, then all correct processes will eventually terminate SRSD.Asg.
- Assignment Totality: if a correct process terminates SRSD.Asg then all correct processes will eventually terminate SRSD.Asg.
- Randomness: the return value of SRSD.Rec follows the uniform distribution.
- Secrecy: if no correct process has called SRSD.Rec then the adversary has no information about the return value of SRSD.Rec.
- Reconstruction Termination: if all correct processes call SRSD.Rec then all correct processes will eventually terminate SRSD.Rec.

redNo Agreement properties ???

Implementation 5 (adapted from [FKT22])**.** The implementation relies on AVSS and BRB. Each process randomly picks a number and shares it among all processes using AVSS. Then, the owner broadcasts a set of processes and all processes reconstruct the secret of these processes and return their sum.

Definition 21. SRSD.Core

The PLTS encoding Algorithm **??** is denoted SRSD.Core. Its adversary has partial information as it should not see the valuation of `x` when it is picked and when AVSS.Sh is called unless the process is Byzantine.

Specification 5. SRSD.Spec

The PLTS encoding of SRSD is provided in Transition System **??**. It is called SRSD.Spec and its adversary is omniscient. Similarly to WCC, Secrecy is encoded by adding a guess transition which allows the adversary to predict the outcome of SRSD.Rec. Secrecy is equivalent to the probability that the adversary's prediction is correct being less than $|X|^{-1}$.

Definition 22. SRSD.Impl

SRSD:Core, BRB:Impl, AVSS:Spec SRSD.Impl is defined as the parallel composition of SRSD.Core, BRB.Impl and AVSS.Spec synchronised on calls, returns and failures from which the API of BRB and AVSS have been abstracted. The adversary of BRB.Impl is omniscient and the adversary of SRSD.Core is omniscient on its interface with BRB.Impl. Furthermore, the adversaries of SRSD.Core and AVSS.Spec agree on their interface. Thus, the composed adversary is well-defined.

Definition 23. SRSD.Hybrid

SRSD:Core, BRB:Spec, AVSS:Spec SRSD.Hybrid is defined as the parallel composition of SRSD.Core, BRB.Spec and AVSS.Spec synchronised on calls, returns and failures from which the API of BRB and AVSS have been abstracted. The adversary of BRB.Spec is omniscient and the adversary of SRSD.Core is omniscient on its interface with BRB.Spec. Furthermore, the adversaries of SRSD.Core and AVSS.Spec agree on their interface. Thus, the composed adversary is well-defined.

0.2.6 Byzantine Reliable Broadcast (BRB)

Each instance of BRB has a designated *leader* with an input message m and processes may return a received message. A program implements BRB if it satisfies three properties: Validity, Totality and Agreement.

- Validity: if the leader is not corrupted then all correct processes must return its input.
- Agreement: if two correct processes return then they return the same value.
- Totality: if a correct process returns then all correct processes eventually return.

Implementation 6 (taken from [Bra87])**.** This is the classical implementation of BRB composed of three rounds of message exchanges relying on quorum intersection. The pseudo-code is provided in Algorithm **??**.

Definition 24. BRB.Impl

The PLTS encoding Algorithm **??** is denoted BRB.Impl. Its adversary is omniscient.

Specification 6. BRB.Spec

This specification contains linear properties only, thus, the encoding as a PLTS is trivial. It can be found in Transition System **??**. We call it BRB.Spec and its adversary is omniscient. Since the leader initiates its instance of BRB, we use only one *call* transition.

0.2.7 Asynchronous Verifiable Secret Sharing (AVSS)

The AVSS is actually a pair of protocols: Sharing Sh and Reconstruction Rec. Each instance has a designated process called the dealer which is known by all processes. The dealer starts Sh with a secret value which it divides into shares and hands one to each process. During Rec, processes gather shares from other processes to reconstruct the original secret. A program implements an ϵ-correct AVSS if it satisfies three properties: Termination, Correctness, Secrecy.

- Termination:
  - if the dealer is correct and has called AVSS.Sh then, with probability $1-\epsilon$, all correct processes will eventually terminate AVSS.Sh;
  - if a correct process has terminated AVSS.Sh then all correct processes will eventually terminate AVSS.Sh;
  - with probability $1-\epsilon$, if all correct processes have terminated AVSS.Sh and called AVSS.Rec then all correct processes will eventually terminate AVSS.Rec.
- Correctness: Once a correct process has terminated AVSS.Sh, there exists $x \in X$ such that, with probability $1-\epsilon$,
  - if the dealer is correct then it has called AVSS.Sh with x;
  - no correct process will terminate AVSS.Rec with a value different from x.
- Secrecy: if the dealer is correct and no correct process has called AVSS.Rec then the adversary has no information about the future retrieved value.

Some properties always hold while others hold with probability $1-\epsilon$. The specification of [CR93] states that all properties hold with probability $1-\epsilon$, but it seems that the specification can be strengthened. In particular, the second point of Termination holds deterministically because it is a consequence of the Totality of BRB.

Our proof will stop at this level of abstraction, meaning that all considered implementations will be correct up to the correctness of their underlying AVSS. Efficient implementations of AVSS enter the realm of cryptography, which is not the main target of our methodology. A possible implementation could have been the one from [CR93].

Specification 7. AVSS.Spec

The PLTS encoding of AVSS is provided in Transition System **??**. We call it AVSS.Spec and its adversary should not see the secret input unless the dealer is Byzantine. To encode the commitment as a linear property, the committed value is provided when AVSS.Sh returns. It uses the same trick as WCC and SRSD to encode secrecy. Since secret values are contained in the interface of AVSS, this specification must be expressed with partial information.

0.3 Proof Decomposition

The end goal is the correctness of ABA.Impl. We establish it via a simulation by ABA.Spec and transport the spec's correctness. The simulation itself is decomposed recursively along the sub-protocol structure.

For a protocol P with concrete sub-protocols $Q_1, \ldots, Q_k$, the step "P.Spec simulates P.Impl" is split into:

1. (Substitution) $\mathsf{P.Impl}[\mathsf{Q_1.Spec}, \ldots, \mathsf{Q_k.Spec}]$ is simulated by P.Impl, using the child simulations Q_i.Spec simulates Q_i.Impl and compositionality.
2. (Core) P.Spec simulates the hybrid $\mathsf{P.Impl}[\mathsf{Q_1.Spec}, \ldots, \mathsf{Q_k.Spec}]$.
3. (Transitivity) Compose (1) and (2).

Leaves of the recursion (no sub-protocol, or only the black-box AVSS.Spec) skip the substitution step.

0.3.1 Top-level

Theorem 9. *ABA.Correct ABA:Impl, ABA:Spec ABA.Impl satisfies Validity, Agreement and Almost-Sure Termination.*

Proof. thm:ABASim,thm:Segala According to Theorem 10, $\mathsf{ABA.Impl} \sqsubseteq^f \mathsf{ABA.Spec}$. Moreover, one can prove that ABA.Spec satisfies all three properties which can all be expressed as trace properties or fair trace distribution properties. Thus, by Theorem 3, ABA.Impl satisfies all three properties. □

0.3.2 ABA

Theorem 10. *ABA.Sim ABA:Impl, ABA:Spec* $\mathsf{ABA.Impl} \sqsubseteq^f \mathsf{ABA.Spec}$.

Proof. lem:ABASubst, lem:ABACore Trivial by transitivity of $\sqsubseteq^f$ and Lemma 12 and 11. □

Lemma 11. *ABA.SubstSim ABA:Impl, ABA:Hybrid* $\mathsf{ABA.Impl} \sqsubseteq^f \mathsf{ABA.Hybrid}$.

Proof. thm:GBCASim, thm:WCCSim, lem:precong Before abstraction, both systems only differ by replacing GBCA.Impl (resp. WCC.Impl) by GBCA.Spec (resp. WCC.Spec). However, ABA.Impl is partial-information so we need to relate its belief PLTS. According to Theorem 8, the belief PLTS of the composition is isomorphic to the composition of the belief PLTS. Therefore, according to Theorem 16 and 13, Lemma 1 followed by Lemma 2 imply $\mathsf{ABA.Impl} \sqsubseteq^f \mathsf{ABA.Hybrid}$. □

Lemma 12. *ABA.CoreSim ABA:Hybrid, ABA:Spec* $\mathsf{ABA.Hybrid} \sqsubseteq^f \mathsf{ABA.Spec}$.

Proof. thm:Segala Both systems are full-information and probabilistic, thus, according to Theorem 3, it suffices to build a progressive probabilistic forward simulation of ABA.Hybrid by ABA.Spec. [TODO details about the construction] □

0.3.3 WCC

Theorem 13. *WCC.Sim WCC:Impl, WCC:Spec* $\mathsf{WCC.Impl} \sqsubseteq^f \mathsf{WCC.Spec}$.

Proof. lem:WCCSubst, lem:WCCCore Trivial by transitivity of $\sqsubseteq^f$ and Lemma 15 and 14. □

Lemma 14. *WCC.SubstSim WCC:Impl, WCC:Hybrid* $\mathsf{WCC.Impl} \sqsubseteq^f \mathsf{WCC.Hybrid}$.

Proof. thm:SRSDSim, thm:GBCASim, lem:precong Before abstraction, both systems only differ by replacing SRSD.Impl (resp. Gather.Impl) by SRSD.Spec (resp. Gather.Spec). However, WCC.Impl is partial-information because of SRSD.Impl, thus, we need to relate the belief PLTS of WCC.Impl to WCC.Hybrid. According to Theorem 8, the belief PLTS of WCC.Impl is isomorphic to the composition of the belief PLTS of its components. Therefore, according to Theorem 20 and 17, Lemma 1 followed by Lemma 2 imply $\mathsf{WCC.Impl} \sqsubseteq^f \mathsf{WCC.Hybrid}$. □

Lemma 15. *WCC.CoreSim WCC:Hybrid, WCC:Spec* $\mathsf{WCC.Hybrid} \sqsubseteq^f \mathsf{WCC.Spec}$.

Proof. thm:Segala Both systems are full-information and probabilistic, thus, according to Theorem 3, it suffices to build a progressive probabilistic forward simulation of WCC.Hybrid by WCC.Spec. [TODO details about the construction] □

0.3.4 GBCA

Theorem 16. *GBCA.Sim impl:GBCA, GBCA:Spec* $\mathsf{GBCA.Impl} \sqsubseteq^f \mathsf{GBCA.Spec}$.

Proof. lem:weaksim Since GBCA is a full-information non-probabilistic protocol, according to Lemma 4, it suffices to build a progressive weak simulation of GBCA.Impl by GBCA.Spec. [TODO details about the construction] □

0.3.5 Gather

Theorem 17. *Gather.Sim Gather:Impl, Gather:Spec* Gather.Impl $\sqsubseteq^f$ Gather.Spec.

Proof. lem:GatherSubst, lem:GatherCore Trivial by transitivity of $\sqsubseteq^f$ and Lemma 19 and 18. □

Lemma 18. *Gather.SubstSim Gather:Impl, Gather:Hybrid* Gather.Impl $\sqsubseteq^f$ Gather.Hybrid.

Proof. thm:BRBSim, lem:precong Before abstraction, both systems are full-information and differ by replacing BRB.Impl by BRB.Spec. According to Theorem 23, Lemma 1 followed by Lemma 2 imply Gather.Impl $\sqsubseteq^f$ Gather.Hybrid. □

Lemma 19. *Gather.CoreSim Gather:Hybrid, Gather:Spec* Gather.Hybrid $\sqsubseteq^f$ Gather.Spec.

Proof. lem:weaksim Since Gather is a full-information non-probabilistic protocol, according to Lemma 4, it suffices to build a progressive weak simulation of Gather.Hybrid by Gather.Spec. [TODO details about the construction] □

0.3.6 SRSD

Theorem 20. *SRSD.Sim SRSD:Impl, SRSD:Spec* SRSD.Impl $\sqsubseteq^f$ SRSD.Spec.

Proof. lem:SRSDSubst, lem:SRSDCore Trivial by transitivity of $\sqsubseteq^f$ and Lemma 22 and 21. □

Lemma 21. *SRSD.SubstSim SRSD:Impl, SRSD:Hybrid* SRSD.Impl $\sqsubseteq^f$ SRSD.Hybrid.

Proof. thm:BRBSim,thm:partialinfocompo Before abstraction, both systems only differ by replacing BRB.Impl by BRB.Spec. However, they are partial-information, thus, we need to relate their belief PLTSs. According to Theorem 8, the belief PLTS of the composition is isomorphic to the composition of the belief PLTSs. Therefore, according to Theorem 23, Lemma 1 followed by Lemma 2 imply SRSD.Impl $\sqsubseteq^f$ SRSD.Hybrid. □

Lemma 22. *SRSD.CoreSim SRSD:Hybrid, SRSD:Spec* SRSD.Hybrid $\sqsubseteq^f$ SRSD.Spec.

Proof. thm:partialinfosound, thm:Segala SRSD.Hybrid is partial-information because SRSD.Impl and AVSS.Spec also are. However, SRSD.Spec is full-information. Thus, according to Theorem 7 and 3, it suffices to build a progressive probabilistic forward simulation of the belief PLTS of SRSD.Hybrid by SRSD.Spec. [TODO details about the construction] □

0.3.7 BRB

Theorem 23. *BRB.Sim impl:BRB, BRB:Spec* BRB.Impl $\sqsubseteq^f$ BRB.Spec.

Proof. lem:weaksim Since BRB is a full-information non-probabilistic protocol, according to Lemma 4, it suffices to build a progressive weak simulation of BRB.Impl by BRB.Spec. [TODO details about the construction] □

Bibliography

[ABDY22] Ittai Abraham, Naama Ben-David, and Sravya Yandamuri. Efficient and adaptively secure asynchronous binary agreement via binding crusader agreement. In *41st ACM Symposium on Principles of Distributed Computing*, pages 381–391, 2022.

[AFW25] Hagit Attiya, Itay Flam, and Jennifer L. Welch. Beyond canonical rounds: Communication abstractions for optimal byzantine resilience. *CoRR*, abs/2510.04310, 2025.

[Bra87] Gabriel Bracha. Asynchronous byzantine agreement protocols. *Information and Computation*, 75(2):130–143, 1987.

[CR93] Ran Canetti and Tal Rabin. Fast asynchronous byzantine agreement with optimal resilience. In *Proceedings of the Twenty-Fifth Annual ACM Symposium on Theory of Computing*, STOC '93, page 42–51, New York, NY, USA, 1993. Association for Computing Machinery.

[DSW22] Brijesh Dongol, Gerhard Schellhorn, and Heike Wehrheim. Weak Progressive Forward Simulation Is Necessary and Sufficient for Strong Observational Refinement. In Bartek Klin, Sławomir Lasota, and Anca Muscholl, editors, *33rd International Conference on Concurrency Theory (CONCUR 2022)*, volume 243 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 31:1–31:23, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[FKT22] Luciano Freitas, Petr Kuznetsov, and Andrei Tonkikh. Distributed randomness from approximate agreement, 2022.

[LT26] Nancy A Lynch and Mark R Tuttle. An introduction to input/output automata. *Form. Asp. Comput.*, January 2026. Just Accepted.

[Seg95a] Roberto Segala. A compositional trace-based semantics for probabilistic automata. In Insup Lee and Scott A. Smolka, editors, *CONCUR '95: Concurrency Theory*, pages 234–248, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg.

[Seg95b] Roerto Segala. *Modeling and verification of randomized distributed real-time systems.* Phd thesis, Massachusetts Institute of Technology, 1995. Available at `https://dspace.mit.edu/entities/publication/e8356692-bb30-4359-9599-165fad91acd2`.

.1 Implementation's Pseudo-code